

Types of RNN | Many to Many | One to Many | Many to One RNNs

➤ What's an RNN?

An **RNN (Recurrent Neural Network)** processes **sequential data** — data where **order matters**, like:

- Sentences (“I love deep learning”)
- Time series (stock prices, weather)
- Music or audio signals

RNNs remember information from previous steps using a **hidden state**, making them ideal for **sequence-based problems**.

➤ Why Different Types of RNN?

Not all sequence tasks are the same!

Depending on **input and output length**, we use different RNN architectures:

- Some tasks have **one input and one output**
- Some have **one input and many outputs**
- Some have **many inputs and one output**
- Some have **many inputs and many outputs**

So, let's go through these one by one

1. One-to-One (Basic RNN / Vanilla Neural Network)

Definition:

Each input corresponds to exactly one output.

Example Tasks:

- Image classification (input = one image, output = one label)
- Simple regression (input = one value, output = one value)

Conceptually:

Input (x) → Model → Output (y)

This is just a **normal feedforward neural network (ANN)**, no sequence here.

We include it for comparison.

2. One-to-Many

Definition:

One input produces a **sequence** of outputs.

Example Tasks:

- **Image captioning**
(Input = one image, Output = sequence of words)
- **Music generation from a prompt**

Example:

Input (x) = image of a cat

Output ($y_1, y_2, y_3 \dots$) = ["A", "cat", "is", "sleeping"]

Flow:

Input $x_1 \rightarrow \text{RNN} \rightarrow h_1 \rightarrow \text{Output } y_1$

$\searrow \rightarrow h_2 \rightarrow \text{Output } y_2$

$\searrow \rightarrow h_3 \rightarrow \text{Output } y_3 \dots$

RNN generates outputs **step by step**, based on its previous hidden states.

3. Many-to-One

Definition:

Multiple inputs (a sequence) lead to **one output**.

Example Tasks:

- **Sentiment analysis:**
Input = sequence of words (a review)
Output = one sentiment (positive/negative)
- **Stock prediction:**
Input = past 10 days' prices
Output = next day's trend (up/down)
- **Speech recognition:**
Input = audio signal sequence
Output = recognized word

Example:

Input = ["I", "love", "this", "movie"]

Output = "Positive"

Flow: $x_1 \rightarrow h_1$ $x_2 \rightarrow h_2$ $x_3 \rightarrow h_3$ $x_4 \rightarrow h_4 \rightarrow \text{Output } y$

Here, only the **final hidden state (h_4)** is used to make the prediction.

Intuition: The network reads the whole sequence, remembers it, and gives one final decision.

4. Many-to-Many

Definition:

A sequence of inputs produces a sequence of outputs.

Example Tasks:

- **Language translation:**
Input = “I love AI”
Output = “J’aime l’intelligence artificielle”
- **Speech-to-text:**
Input = audio frames
Output = text sequence
- **Video classification (frame by frame)**

Flow: $x_1 \rightarrow h_1 \rightarrow y_1$ $x_2 \rightarrow h_2 \rightarrow y_2$ $x_3 \rightarrow h_3 \rightarrow y_3$

Each input produces an output, maintaining the same time steps.

➤ Two Variants of Many-to-Many:

1. Equal-length Many-to-Many

- Input and output sequences are the same length
- Example: POS tagging, where each word gets one tag (“I” → PRON, “love” → VERB, “dogs” → NOUN)

2. Unequal-length Many-to-Many

- Input and output lengths differ
-

- **Example:** Machine translation
(English sentence → French sentence, lengths may differ)

This is usually done using **Encoder-Decoder architectures**, where:

- **Encoder** reads the input sequence and summarizes it in a **context vector**.
- **Decoder** then generates the output sequence.

5. Variable Length Sequences

Definition:

Both input and/or output lengths can vary real-world data isn't always fixed size.

Examples:

- Translation: input = 5 words, output = 7 words
- Speech: variable-length audio clips

How RNN Handles It:

- Padding or truncating sequences
- Using special “end” tokens (<EOS>)
- Encoder-decoder models (to handle flexible lengths)

➤ Summary Table

Type	Input	Output	Example
One-to-One	Single	Single	Image classification
One-to-Many	Single	Sequence	Image captioning
Many-to-One	Sequence	Single	Sentiment analysis
Many-to-Many	Sequence	Sequence	Machine translation
Variable Length	Variable	Variable	Real-world sequences

➤ Intuitive Way to Remember

Category	Think of it as...
One-to-One	Simple ANN
One-to-Many	“Tell me a story from one image”
Many-to-One	“Summarize this sentence”
Many-to-Many	“Translate sentence A into B”
Variable Length	“Flexible — can handle any size of input/output”

➤ Example (Keras Implementation Overview)

You can design these types easily using Keras RNN layers:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

# Example: Many-to-One
model = Sequential([
    SimpleRNN(128, input_shape=(timesteps, features)),
    Dense(1, activation='sigmoid')
])
Or for Many-to-Many (sequence output):
from tensorflow.keras.layers import TimeDistributed

model = Sequential([
    SimpleRNN(128, return_sequences=True, input_shape=(timesteps, features)),
    TimeDistributed(Dense(1, activation='sigmoid'))
])
```

➤ Summary (Easy Notes for Revision)

- **RNNs** process sequential data — keeping memory of past steps.
 - **Different types** depend on how many inputs/outputs are involved.
 - **Many-to-One:** Summarize a sequence (e.g., sentiment)
 - **One-to-Many:** Generate a sequence (e.g., captions)
 - **Many-to-Many:** Sequence in, sequence out (e.g., translation)
 - **Variable length:** Real-world data — handled using encoders and decoders.
-